

Critic-Matched Movie Prediction

All-time random holdouts with analytic, boosted-tree, and neural modes

Portfolio project — technical report

Rotten Tomatoes critic reviews (Kaggle, ~1.4M rows through early 2023)

Abstract

We simulate user histories from critics instead of restricting evaluation to a late temporal window. For each fake user, sampled ratings are seen movies and a different rated movie is an unseen target. The analytic formula centers on the target movie’s critic mean, applies signed Pearson similarity, and maps critic rating magnitude to the user. Evaluation is *paired and nested*: a fixed target and a fixed seen-order per pseudo-user are reused across every seen-count, so the seen-independent baselines are exactly flat and the curves are smooth. Across 43,344 paired episodes from 301 test critics, full-history analytic RMSE is 0.796 versus 0.826 for the reviewer mean and 0.830 for a calibrated Tomatometer. Two trained models — an XGBoost regressor (0.775) and a deeper residual neural-network ensemble (0.773) — beat every baseline at every seen-count and are statistically tied with each other. The gain is calibration, not taste matching: both models lean first on the movie’s Tomatometer, then the user’s mean. A fully parallel second project (§7) repeats the entire pipeline on 11.08M real Letterboxd community ratings (1–10 scale, dense per-member histories instead of sparse critic pseudo-users); normalizing both projects’ RMSE by rating range shows they are comparable, with Rotten Tomatoes marginally *better* normalized despite its sparser profiles. Both projects also train a parallel z-score track — each rater standardized to their own scale, predicting pure deviation rather than level — to directly test whether the gain above is calibration or genuine taste-matching; the result is not uniform across the two datasets, reported as found rather than presupposed (§8).

1 Data and score scale

The source is `andrezaza/clapper-massive-rotten-tomatoes-movies-and-reviews`. Fractions, letter grades, percentages, and star ratings are parsed and quantized to $\{0, 1, 2, 3, 4, 5\}$, called the *standardized raw score*. It is the prediction target. Critics with at most five parseable scores are excluded, name variants are deduplicated, and repeat critic–movie cells are averaged.

Data funnel	Count
Raw review rows (start)	1,444,963
Distinct critics with a parseable score	8,623
After the low-volume critic floor (> 5 scored reviews)	4,393 critics, 1,262,747 rows
Parsed, standardized scored reviews used for modelling	992,954
Distinct movies with any scored review	58,736
Neighbour pool / pseudo-users (≥ 10 distinct movies)	3,704 critics
Movies in the final app catalog (most-reviewed)	1,000
Critics appearing in that catalog	3,387

Table 1: From 1.4M raw rows to the modelling pool and the app catalog.

The data retain a diagnostic z-score:

$$z_{c,i} = \frac{r_{c,i} - \mu_c}{\sigma_c}. \quad (1)$$

Here $r_{c,i}$ is critic c 's standardized raw score for movie i ; μ_c is critic c 's all-time mean score; and σ_c is critic c 's all-time standard deviation. Thus $z_{c,i} = 1$ means one critic-specific standard deviation above the critic's usual score. The main designs (§6) use raw scores, not z-scores, because rating magnitude is part of the analytic formula; §8 trains a parallel z-score variant of every design as a direct test of whether that magnitude information is doing real work.

The *Tomatometer* is the percentage of reviews labelled Fresh, not a mean 0–5 rating. Baseline B2 fits $t_m = aT_m + b$, where T_m is movie m 's Tomatometer percentage, a is a fitted slope, b is a fitted intercept, and t_m is the mapped 0–5 score.

2 All-time random-holdout pseudo-users

The all-time matrix has 3,715 critics with at least 10 scored reviews; 3,704 have 10 distinct movies and can form episodes (the floor was lowered from 20 to widen the pool). Critic identities are split into 2,592 train, 556 validation, and 556 test critics. No trained model sees a validation or test critic as a pseudo-user during fitting.

Training episodes are unpaired for variety: sample n of pseudo-user u 's rated movies as seen ratings, then choose a different rated movie as target m (at least three other reviews; u excluded from target aggregates), for $n \in \{3, 5, 10, 20, 50, \text{all}\}$.

Test episodes are *paired and nested* so the comparison across seen-counts is clean. Sampling a fresh random target per n made even the seen-independent baselines wobble column to column. Instead, for each test pseudo-user with more than 50 movies, 8 targets are fixed once; for each target and each of 3 redraws, the remaining movies are placed in one popularity-weighted order, and the seen set at seen-count n is the first n of that order (all for $n = \text{all}$). A single deterministic routine yields these 41,472 episodes, and every model is scored on byte-identical (u, m, n, draw) keys. Because the target and seen order are fixed across n , every seen-count and every baseline is scored on the same films.

This is item-holdout evaluation, not chronological forecasting: a target can predate a seen movie. It enables catalog prediction for unseen movies but does not claim to predict unreleased films.

3 Pearson similarity and magnitude

Let $O_{u,c}$ be movies rated by pseudo-user u and critic c among the sampled seen movies. Pearson correlation is

$$\rho_{u,c} = \frac{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})(r_{c,i} - \bar{r}_{c,O})}{\sqrt{\sum_{i \in O_{u,c}} (r_{u,i} - \bar{r}_{u,O})^2} \sqrt{\sum_{i \in O_{u,c}} (r_{c,i} - \bar{r}_{c,O})^2}}. \quad (2)$$

In Equation 2, i indexes a shared seen movie; $r_{u,i}$ and $r_{c,i}$ are user and critic raw scores; $\bar{r}_{u,O}$ and $\bar{r}_{c,O}$ are their respective means over $O_{u,c}$; and $\sum$ adds over the overlap. $\rho_{u,c}$ ranges from -1 for opposite score movement to $+1$ for matching movement. The square roots normalize covariance by each overlap score spread.

Small overlaps are shrunk toward zero:

$$s_{u,c} = \rho_{u,c} \frac{\min(n_{u,c}, k)}{k}. \quad (3)$$

Here $s_{u,c}$ is signed shrunken similarity, $n_{u,c} = |O_{u,c}|$ is overlap count, $\min$ selects the smaller input, and validation selected $k = 8$. $|s_{u,c}|$ is absolute, nonnegative similarity magnitude.

The rating-magnitude multiplier is

$$g_{u,c} = \frac{\sum_{i \in O_{u,c}} r_{u,i} r_{c,i}}{\sum_{i \in O_{u,c}} r_{c,i}^2}. \quad (4)$$

$g_{u,c}$ is `mag_sim` in code. It is a least-squares through-origin mapping from critic to user ratings. If $r_{u,i} = 1.25r_{c,i}$ on every overlap movie, then $g_{u,c} = 1.25$.

4 Movie-mean-centered analytic prediction

For target movie m , let C_m be other critics who rated it. Its movie mean is

$$\bar{r}_m = \frac{\sum_{c \in C_m} r_{c,m}}{|C_m|}. \quad (5)$$

Here $r_{c,m}$ is critic c 's target score, $|C_m|$ is the other-reviewer count, and $\bar{r}_m$ is the mean rating for movie m , not a career mean or Tomatometer percentage.

The requested prediction is

$$\hat{r}_{u,m} = \frac{\sum_{c \in C_m} (|s_{u,c}| \bar{r}_m + s_{u,c}(r_{c,m} - \bar{r}_m)) g_{u,c}}{\sum_{c \in C_m} |s_{u,c}|}. \quad (6)$$

In Equation 6, $\hat{r}_{u,m}$ is the predicted 0–5 score; $s_{u,c}$ comes from Equation 3; $g_{u,c}$ comes from Equation 4; and movie terms come from Equation 5. The denominator intentionally contains only $|s_{u,c}|$, not $g_{u,c}$. Predictions are clipped to $[0, 5]$; a zero denominator falls back to $\bar{r}_m$.

5 Learned models and metrics

Two models share one 38-column feature set, and *both use all of it*: for each target reviewer, the reviewer score minus that critic's leave-one-target all-time mean; reviewers ranked by similarity and their similarity-times-deviation values summarized by mean, count, and standard deviation in ten deciles; then a tail of seen count, overlap statistics, mapped Tomatometer, reviewer count, dispersion, genre, and user mean.

Design 2 is an XGBoost regressor; it handles a missing Tomatometer natively. *Design 3* is a residual neural network in PyTorch, trained on the Apple GPU (MPS). The 37 numeric features (log1p-compressed where they are counts, NaN-imputed and standardized on train statistics) and a 24-dimensional genre embedding feed a projection into six pre-activation residual blocks of width 512 ($\approx 3M$ weights; batch-norm, GELU, dropout), then a linear head. It is trained with AdamW, a ReduceLROnPlateau schedule, and early stopping; three independently seeded networks are averaged into an ensemble, over 32 fake-user profiles per critic and seen-count ($\approx 437k$ rows). Full-history training rows and the log compression are both needed so the net does not extrapolate at

$n = \text{all}$, where the seen count far exceeds any finite- n training row; the trees are immune to this. Both are verified leakage-free: peer means are leave-one-out and u is excluded everywhere. The same feature code powers training and app inference.

B1 is the all-time global mean; B2 is mapped Tomatometer with reviewer-mean fallback; B3 is $\bar{r}_m$; B4 is an unweighted mean of up to ten positively aligned reviewers. Primary error is

$$\text{RMSE} = \sqrt{\frac{\sum_{j=1}^N (\hat{r}_j - y_j)^2}{N}}, \quad (7)$$

where j indexes an episode, $\hat{r}_j$ is a prediction, y_j is its held-out score, and N is episode count. Lower values are better.

6 Results

Seen ratings n	3	5	10	20	50	all
B1 global mean	1.049	1.049	1.049	1.049	1.049	1.049
B2 Tomatometer $\rightarrow$ score	0.830	0.830	0.830	0.830	0.830	0.830
B3 reviewer mean	0.826	0.826	0.826	0.826	0.826	0.826
B4 top-10 similar mean	0.908	0.896	0.881	0.865	0.856	0.845
Design 1 formula	0.964	0.922	0.864	0.833	0.811	0.796
Design 2 XGBoost	0.815	0.804	0.795	0.788	0.780	0.775
Design 3 neural net	0.815	0.804	0.795	0.786	0.778	0.773

Table 2: RMSE on the 43,344 paired, nested episodes. Baselines are flat because they do not depend on the seen set.

Design 1 improves over B3 by 3.6% and B2 by 4.1% at full history, but the two trained models win everywhere: XGBoost 0.792 overall / 0.775 full history, the neural net 0.792 / 0.773. Scaling the network up — wider (512), deeper (six residual blocks), longer training, and doubled fake-user training data — to look for a double-descent gain moved the loss by nothing, and XGBoost itself was unchanged by the extra data: on this engineered tabular feature set the ceiling is the information in the features, not model capacity. Both models improve only gently with n because their dominant feature is the movie’s Tomatometer ($\approx 11\times$ the next feature in XGBoost gain), then the user’s mean; this is why the learned loss is as low as it is, and it was checked to be a leakage-free calibrated consensus, not memorization.

Formula component (full-history episodes)	RMSE
Reviewer mean / movie consensus	0.850
Magnitude-scaled movie consensus	0.833
Movie-centered signed similarity ($g_{u,c} = 1$)	0.867
Full Design 1 formula	0.850

Table 3: Formula attribution: magnitude scaling supplies the whole gain over consensus.

Magnitude matching supplies the entire improvement over the consensus; the signed similarity term adds nothing on top. As with the trained models, correlation-based taste matching contributes little — the useful personalization is scale/level calibration. Across low, middle, and high critic-disagreement terciles, full-history gain over B2 is $+5.7/+6.2/+1.5\%$ for Design 1, $+7.5/+7.4/+5.6\%$



Figure 1: Error versus sampled seen history. With paired evaluation the baselines are exactly flat; Design 1 falls smoothly from 0.964 to 0.796 and crosses them near $n = 50$. Both trained models beat every baseline at every seen-count and overlap each other.

for XGBoost, and +7.5/+7.4/+5.9% for the neural net — all three help in every tercile, the trained models most.

7 Second dataset: Letterboxd community ratings

To test whether real, dense per-user rating histories beat the sparse critic pseudo-users above, we built a fully parallel second project on Letterboxd community ratings (Kaggle, ~11.08M ratings), isolated in its own directory tree (`src/letterboxd/`, `data/letterboxd/`, `results/letterboxd/`) that never shares data, models, or code with the Rotten Tomatoes project. The export yields 7,420 members with at least five rated films across 286,069 films — 11,078,045 ratings total, not 11 million distinct members. Ratings are native on a 1–10 scale (no score parsing). All three designs adopt the *same* 37-feature similarity-decile contract as Design 2/3 above, minus the Tomatometer feature (Letterboxd has no critic-consensus meter): ten deciles of similarity-weighted deviation, plus seen count, overlap statistics, rater count, dispersion, genre, and the member’s own mean. Design 3 is a genuinely trained residual MLP ensemble, identical in architecture to the Rotten Tomatoes network, and — being inductive over engineered features rather than a member/movie embedding — it scores a brand-new member’s live ratings in the browser, just like Designs 1 and 2.

Honest cross-dataset finding. Raw RMSE is not comparable across the two projects because the target scales differ (0–5 versus 1–10); normalizing by rating range ($\text{RMSE} / (\text{max} - \text{min})$) puts them on the same footing. Rotten Tomatoes’ normalized RMSE is 0.159 / 0.155 / 0.155 (Design 1/2/3); Letterboxd’s is 0.167 / 0.167 / 0.169. The two are *comparable*, with Rotten Tomatoes marginally *better* normalized despite training on far sparser critic pseudo-user profiles instead of

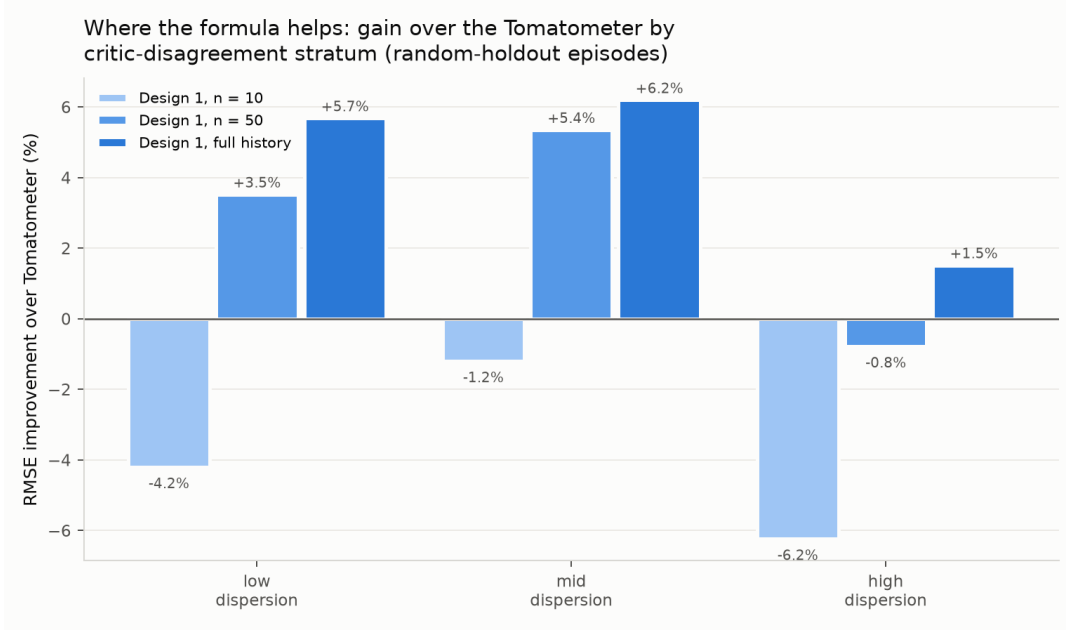


Figure 2: Design 1 gain over B2 by target-movie critic disagreement.

Seen ratings n	3	5	10	20	50	all
B1 global mean	2.108	2.108	2.108	2.108	2.108	2.108
B3 mean of all members	1.639	1.639	1.639	1.639	1.639	1.639
B4 top-10 similar mean	1.705	1.686	1.671	1.653	1.657	1.622
Design 1 analytic	1.765	1.670	1.597	1.561	1.545	1.507
Design 2 XGBoost	1.662	1.612	1.594	1.569	1.536	1.502
Design 3 neural net	1.672	1.621	1.596	1.562	1.545	1.518

Table 4: Letterboxd RMSE (1–10 scale) on the same paired, nested seen-history protocol as Table 2: 300 deterministic test members, 8 fixed targets each, 3 seen-order redraws.

Letterboxd’s dense real member histories. This is reported as found: more (real) rating data did not translate into a lower normalized error at this scale and with this feature contract. Letterboxd’s genuine value is dense, real per-member histories rather than synthetic pseudo-users — not a lower headline RMSE. Full numbers are in `results/letterboxd/cross_dataset_comparison.csv`.

8 Isolating scale: the z-score track

The gain above is mostly calibration — knowing a rater’s own level and the movie’s consensus level — rather than similarity-based taste-matching. To test that claim directly rather than infer it, both projects also train a parallel z-score variant of every design: each rater (fake user or real member) is standardized to their own scale using *seen-set* statistics for the user side and *all-time* statistics for the peer side (§1 defines $z_{c,i}$ for the peer side; the user side uses the mean/std of only the ratings sampled for that episode, since a real visitor only ever has their own seen films to standardize by). The model predicts pure deviation, not level, and every prediction is converted back to the raw scale ($\hat{r} = \mu_{\text{user}} + \sigma_{\text{user}}\hat{z}$, clipped) before scoring, so raw-track and z-track RMSE are directly comparable.

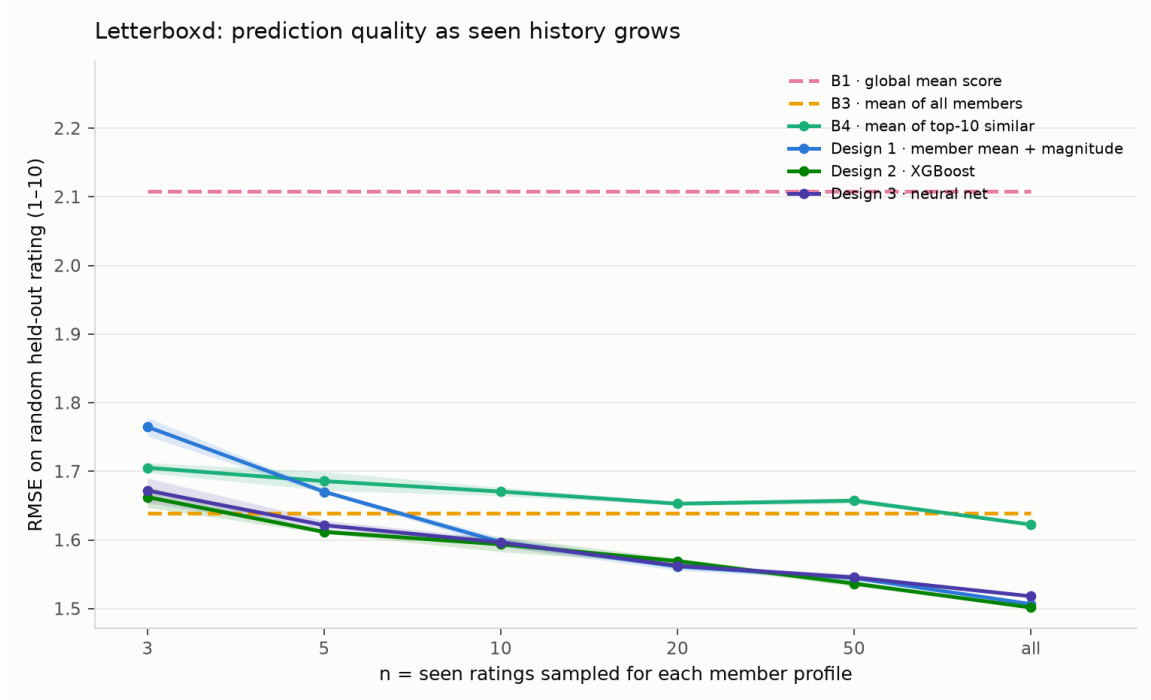


Figure 3: Letterboxd: error versus sampled seen history, styled identically to Figure 1. All three designs converge to ≈ 1.50 – 1.52 at full history.

The result is **not uniform** across the two datasets, and is reported as found rather than presupposed. On Rotten Tomatoes the z-track trails the raw track at every design and every seen-count; removing the Tomatometer-anchored level signal costs more than the deviation-only target gains. On Letterboxd, the trained models (Design 2/3) come out *slightly better* in z-space at full history, while the analytic formula — which has no learned capacity to reallocate — is worse, same as Rotten Tomatoes. A plausible reading: with a dense, purely relational feature contract (no external consensus feature like the Tomatometer) and a trained model with spare capacity, removing each rater’s own level lets that capacity concentrate on the deviation signal instead of re-deriving it; the analytic formula never had spare capacity to reallocate, so it only loses the calibration information without ever regaining it. Either way, the original conclusion stands: even where the z-track modestly *helps* (Letterboxd’s trained models), the gain over raw is under 1.5%, an order of magnitude smaller than what raw calibration bought over the flat baselines in Table 2 — calibration, not taste-matching, remains the dominant effect in this data.

9 Application and limitations

The website (Table 1: 1,000 catalog movies, 3,387 critics) is organized in three sections, and a sort control orders the search lists alphabetically (default), by year, or by most reviewed; the search dropdown scrolls through every match rather than truncating. (1) *Films you have seen*: a search box adds a film to a table where each row carries a five-star widget — click a star and it and all before it light gold, with the rating shown beside it — and the search box clears after each add so films accumulate; rows can be removed. (2) *Films to predict*: the same searchable star-table, but scoring is optional. (3) *Predictions*: for every target film the app shows all three model predictions

Design	raw RMSE	z RMSE (converted back)	z - raw
<i>Rotten Tomatoes (0-5)</i>			
Design 1 analytic	0.796	0.926	+0.130
Design 2 XGBoost	0.775	0.783	+0.008
Design 3 neural net	0.773	0.788	+0.015
<i>Letterboxd (1-10)</i>			
Design 1 analytic	1.507	1.640	+0.134
Design 2 XGBoost	1.502	1.491	-0.010
Design 3 neural net	1.518	1.497	-0.021

Table 5: Full-history RMSE, raw vs. z-score track, both reported on the raw scale (`results/{rotten_tomatoes,letterboxd}/raw_vs_z.csv`).

side by side with the movie’s consensus mean and its Tomatometer. When the visitor scores some target films, the app reports the mean squared error of each method — and of the consensus mean — *against the visitor’s own score*, and marks the closest, answering “which method predicts me best?”. Predictions update live as stars change. A dataset switcher swaps to the Letterboxd project, which is otherwise identical apart from a 1–10 numeric stepper in place of the five-star widget and no Tomatometer column.

Two secondary metrics (`comparison.analysis`, `secondary_metrics.csv`) check that the gain is calibration rather than reordering: within-user Spearman rank correlation between predictions and held-out ratings, and precision@10 (the fraction of a pseudo-user’s predicted top-10 targets that are also in their true top-10 held-out targets). Design 1’s within-user Spearman is 0.578 versus 0.580 for the reviewer mean (B3) — ranking is barely changed, consistent with the attribution finding that the win is calibration, not reordering a user’s films.

Item holdout is not temporal forecasting. Lowering the critic floor to 10 adds noisier low-volume critics. The magnitude multiplier is an unregularized through-origin slope. The neural net matches but does not beat the trees here. Scores are quantized to six levels, pseudo-users are professional critics rather than casual users, and Tomatometer values are current snapshots.

Repository layout

Each Rotten Tomatoes design lives in its own package under `src/rotten_tomatoes/`: `design1_analytic/` (`analytic.py` the formula and top- k baseline, `run.py` the k-sweep and paired test, `attribution.py` the formula-component decomposition, `predict.py` app inference), `design2_xgboost/train.py` (XGBoost training and paired-test evaluation), `design3_neural/` (`network.py` the residual MLP, `train.py` ensemble training), and `comparison/analysis.py` (the figures and tables above). The pseudo-user substrate (`pseudo_users.py`) and the shared 37/38-feature contract (`features.py`) live once at the `rotten_tomatoes` package root and are imported by all three designs, so the comparison is exact by construction rather than by convention. `preprocessing/` ingests and standardizes the raw Kaggle export; `app_catalog/` and `web_export/` build the 1,000-film browser catalog and the client-side model export. `src/letterboxd/` mirrors this layout exactly (its own `features.py`, `train_xgboost.py`, `train_neural.py`, `analyze.py`, `web_export.py`), sharing no code, data, or model artifacts with the Rotten Tomatoes package. `tests/` covers the formula, the random-holdout and paired-episode protocols, partition disjointness, and the feature contract.

Verified in this workspace: unit tests confirm $g_{u,c} = 1.25$ for an exact $r_u = 1.25 r_c$ overlap and

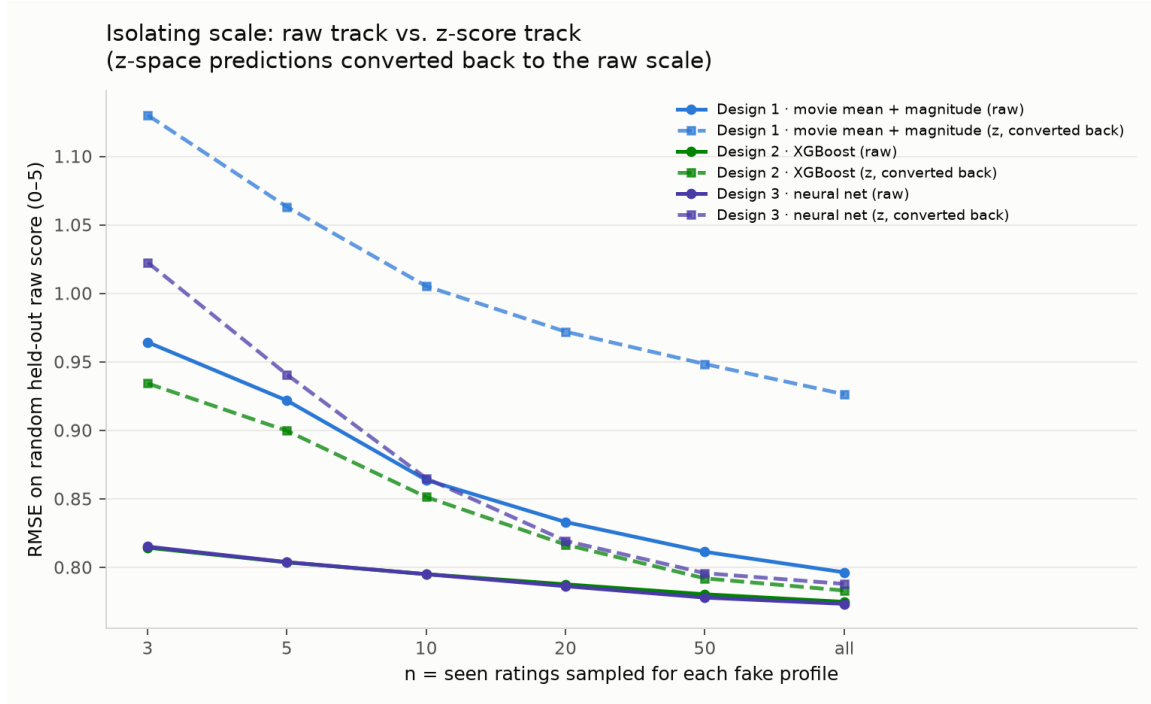


Figure 4: Rotten Tomatoes: raw (solid) vs. z-score (dashed) RMSE by seen count, per design.

the specified numerator/denominator; random-holdout tests prove a sampled target is never in its own seen set; a paired-episode test proves nested seen sets are prefix-nested and deterministic; partition tests prove train/validation/test pseudo-user identities are disjoint and exhaustive; and every design, the attribution script, and the analysis/export pipeline ran end to end from the all-time processed data, with the low XGBoost loss checked to be a leakage-free calibrated consensus (flat per- n baselines, a dominant Tomatometer feature, leave-one-out peer means) rather than memorization.

Reproducibility. Python, pandas, scipy.sparse, NumPy, XGBoost, PyTorch, and matplotlib. The pseudo-user substrate and feature code are shared, seeded modules (see Repository layout above), so the cross-design comparison is exact. The Letterboxd project under `src/letterboxd/` mirrors this layout exactly. Both projects also run entirely client-side in the browser (Vite/React/TypeScript, [web/](#)), hosted as a static GitHub Pages site with no server. Run the ordered module pipelines listed in the README from `src/`; all random seeds are fixed.